

Университет ИТМО

Факультет программной инженерии и компьютерной техники

Лабораторная работа №1

по «Алгоритмам и структурам данных»

Базовые задачи

Выполнил:

Студент группы Р3206

Косогова М. А.

Преподаватели:

Косяков М.С.

Тараканов Д. С.

Санкт-Петербург

2026

Задача №N «Свинки-копилки»

```
#include <iostream>
```

```
using namespace std;
```

```
int* keys;
```

```
int* states;
```

```
int broken_piggy = 0;
```

```
void openbank(int bank) {
```

```
    states[bank] = 2;
```

```
    int nextbank = keys[bank];
```

```
    if (states[nextbank] == 0) {
```

```
        openbank(nextbank);
```

```
    } else if (states[nextbank] == 2) {
```

```
        broken_piggy++;
```

```
    }
```

```
    states[bank] = 1;
```

```
}
```

```
int main() {
```

```
    int N;
```

```
    cin >> N;
```

```
    if (N >= 1 && N <= 100) {
```

```
        keys = new int[N];
```

```
        states = new int[N];
```

```
        for (int i = 0; i < N; i++) {
```

```
            cin >> keys[i];
```

```
            keys[i]--;
```

```
            states[i] = 0;
```

```
        }
```

```
        for (int i = 0; i < N; i++) {
```

```
            if (states[i] == 0) {
```

```

    openbank(i);
}
}
delete[] keys;
delete[] states;
cout << broken_piggy << endl;
} else {
    std::cout << "N read error!" << std::endl;
}
return 0;
}

```

Пояснение к примененному алгоритму:

- 1) Считываем количество копилки и для каждой копилки запоминаем номер копилки, которой лежит ее ключ, а также создаем массив состояний для каждой копилки: 0 – еще не открывали, 1- уже обработали, 2 – сейчас в процессе открытия (в текущей цепочке);
- 2) Для каждой еще не обработанной копилки запускаем рекурсивную функцию (openbank), здесь мы помечаем копилку статусом 2 (в процессе), берем из нее ключа и переходим к следующей копилке по этому ключу;
- 3) Если при переходе мы попадаем на копилку, которая уже имеет статус 2, значит, что в текущей цепочке образовался цикл, и в таком случае мы увеличиваем счетчик разбитых копилки на 1 (одну копилку в цикле придется разбить, чтобы открыть все остальные);
- 4) После завершения обработки цепочки (то есть возврата из рекурсии) помечаем текущую цепочку статусом 1, значит, что вся цепочка, начиная с этой копилки, уже обработана, и повторно ее рассматривать не нужно;
- 5) Перебираем все копилки по очереди, то есть, если копилка еще не открывалась (статус 0), запускаем из нее новую цепочку, и после обработки всех копилки выводим общее количество разбитых копилки.